

GSoC 2025 Proposal

Personal Information

Name: Yeshwanth Reddy Katta

Email: yeshwanthreddykatta2000@gmail.com

GitHub: github.com/yeshwanthkatta

LinkedIn: linkedin.com/in/katta-yeshwanth-reddy

Personal Website: yeshwanthreddykatta.com

Phone: +1 (571) 275-6341

University: George Washington University

Degree Program: Master of Science in Computer Engineering

Expected Graduation: May 2025

Location (during GSoC): Arlington, Virginia, USA (UTC -4 / EDT)

1. Title

Using AI to Improve Open-Source IP

2. Synopsis

As modern hardware design is growing in complexity by Power, performance, and Area, maintaining large and legacy code databases becomes increasingly difficult. The project proposed by Mr. Steve Hoover's focus is converting Verilog to TL-Verilog with the help of AI-assisted automation interfacing with LLMs like CHATGPT. It uses Python script, which drives the conversion flow through incremental refactoring steps and runs FEV (Formal Equivalence Check) to ensure the correctness of the modified code.

3. Benefits to the Community

1. By converting to TL-Verilog, the project promotes cleaner and more modular code structures, aligning with the principles of Steve Hoover. (Maintainability of Open-Source Verilog Code)

2. Improvements in the convert.py script motivate the community to practice automation in hardware design, enabling developers to apply the conversion pipeline to their Verilog Projects. (Reusable Automation Toolchain)

3. Using FEV tools like Yosys, SymbiYosys, and EQY after the refactoring step ensures confidence in the community in the automated conversion flow.

4. One of the Key byproducts of this project is generating a high-quality dataset that contains Prompt-Response Pairs, Prompts given to the LLM, Verilog Code before and after change, FEV Outcome Labels like PASS/FAIL, Timestamps, and retry counts that can be used to train future LLMs in hardware design, Contributing to the broader field of AI-assisted automation. (AI Model Training data)

5. Addressing Compatibility issues encountered on macOS will make the toolchain more accessible to diverse users.

6. One of the most important outcomes of this project is the adoption of TL-Verilog, promoted by the FOSSI Community and Steve Hoover. This brings modularity, abstraction, and time-decoupling benefits. Converting actual Verilog design to TL-Verilog and integrating it into a maker chip IDE provides tutorials and working examples, lowering the barrier to adoption for new learners and institutions.

4. Deliverables

Setup & Exploration (Done)

1. Using MacPorts, installed and built Yosys, SymbiYosys, and EQY on macOS (Apple M2 Pro). Understood the role of FEV tools in open-source Formal Equivalence Verification, ensuring correctness between refactor stages.
2. Contributed https://github.com/yeshwanthkatta/conversion-to-TLV_Copy/blob/yeshwanthkatta-patch-1.0/macOS_installation.md which is in the personal repo.
3. Joined TL-Verilog Slack and interacted with a mentor (Steve Hoover)
4. I reviewed the conversion-to-TLV flow and tested initial prompt runs while exploring the convert.py script and prompts.json flow.
5. Made notes: https://github.com/yeshwanthkatta/conversion-to-TLV_Copy/blob/manual-Conversions/Notes%20on%20Conversion%20Flow_TL%20Verilog.pdf.

Foundational Learning and Early Conversion Work (Completed)

As part of my preparation for this project, I've completed exploring several self-guided materials and exercises recommended by my mentor, Steve Hoover.

1. Studied the Fundamentals of TL-Verilog from Makerchip Tutorials and Labs
2. Explored M5 GitHub repo(<https://github.com/rweda/M5>)
3. Analyzed the convert.py script and made notes.
4. Manually walked through refactoring examples using pen and paper for Mux and Counter, which are simple Verilog Modules, and SPI block, which I implemented in my grad project. (https://github.com/yeshwanthkatta/conversion-to-TLV_Copy/blob/manual-Conversions/SPI%20Conversion%20Readme)

Next Steps:

1. The next steps will run the entire refactoring flow manually (without integrating into the LLM) on simple open-source Verilog Modules.
2. Enhance Prompt Flow for data capture(<https://github.com/stevehoover/conversion-to-TLV/blob/main/prompts.json>)
3. Begin TL- Verilog implementation and then integrate with API key from ChatGPT.
4. Documentation and Tutorials: Publish TL-Verilog-ready refactor examples in Maker Chip

5. Timeline

Phase	Dates	Activities	Deliverables
Community Bonding	May 8 – June 1	- Finalize Verilog modules for refactor (Mux, Counter, SPI, SERV0) - Meet with mentor to review prompt flow and LLM strategy - Review past examples like SERV - Get familiar with Maker chip usage for TLV testing	Checklist of modules to refactor Baseline prompt analysis
Coding Start	June 2	Official GSoC coding phase begins	—
Week 1–2	June 2 – June 15	- Run manual prompt flow on Mux b & Counter using convert.py (no LLM) - Validate each step using EQY - Start structured logging of prompt flows	Logged prompt–input–output–FEV results Initial data set
Week 3–4	June 16 – June 29	- Integrate OpenAI GPT API - Compare LLM vs manual refactors - Score results by FEV and comment handling	LLM output logs Prompt comparison chart Refactored modules with evaluation ready
Week 5–6	June 30 – July 13	- Refactor more complex Verilog (FSM + Memory or Serv module) - Improve automation reliability (fallback, retries) - Test TLV correctness in Maker chip	Full pipeline log of complex module Working TLV versions of all simple designs
Midterm Evaluation	July 14 – July 18	Submit midterm progress to GSoC dashboard Review with mentor	Midterm summary SPI TLV Prompt dataset Tool improvements
Week 7–8	July 19 – August 1	- Continuation of Refactoring more complex Verilog like Serv Module	Refactor of Serv-based module Improved tool with robust logging
Week 9–10	August 2 – August 15	- Finalize and test TLV module - Test in Maker chip - Visualize output using waveform viewer - Document TLV model behavior	Maker chip waveforms Post-refactor test report
Week 11	August 16 – August 22	- Write tutorial: <i>“How I Used convert.py to Refactor Verilog”</i> - Publish public GitHub repo with	Final GitHub repo Blog/Tutorial draft

Phase	Dates	Activities	Deliverables
		datasets, TLV, Verilog, and documentation	
Week 12: Final Submission	August 23 – August 25	- Submit final code and documentation - Upload evaluations to GSoC portal - Mentor evaluation and confirmation	Final submission package GSoC evaluation form

6. Personal Background & Motivation (Why Me)

I am Yeshwanth Reddy Katta, a master's thesis student in Computer Engineering at George Washington University with a focus area of MEMS, Photonics, and Electronics, having a strong foundation in VLSI design, RTL modeling, ASIC testing, and circuit verification through Academic projects. My academic work has been focused on combining low-level hardware understanding through RTL simulation, analog design, or AI-enhanced tooling. Currently, I am working as a Hardware Engineering Intern at OpenQQuantify, a start-up focused on integrating AI into Electronics. My work focuses on generating circuits using LLMS and simulating them in MATLAB. I am completing my internship in April 2025, ready to start GSOC for the next 3 months.

My fascination with digital systems began in my undergraduate studies, where I worked on processors and communication systems. This interest deepened during my graduate work, where I designed and verified a 64-QAM modulator in Verilog for SoC integration and modeled complex analog behavior (such as hysteresis) using Cadence Virtuoso in my Thesis project.

At Bosch, I developed bare-metal firmware for Renesas RH850 microcontrollers and collaborated closely with hardware validation teams—an experience that helped me appreciate the need for precision, clarity, and maintainability in hardware designs.

When I encountered the "Using AI to Improve Open-Source IP" GSoC project, it immediately resonated with my interests. The combination of "Converting Verilog to TL-Verilog using LLms " aligns perfectly with my goals. I've already begun contributing by manually converting Verilog modules to TL-Verilog, writing structured prompt logs, and manually converting my project blocks like the SPI module. I'm excited to continue this work at scale—transforming more Verilog modules into modular, clean, TL-Verilog modules while contributing to the Open-Source Through FOSSI organization

7. Technical Details (Tools Used in this Project)

Yosys	Open-source Verilog synthesis and analysis tool; used for RTL parsing and preparation for FEV.
SymbiYosys (sby)	Script-driven formal verification wrapper that automates equivalence checking.
EQY	Open-source equivalence checker integrated with the flow to compare original and refactored Verilog.
convert.py	Core Python script that applies a series of prompts step-by-step to refactor Verilog.
prompts. json	Contains the prompt "system messages"—each entry defines one transformation step in the refactoring flow.
Meld	Visual diff tool used for reviewing code changes between original and refactored versions.
OpenAI API	Powers the LLM used to automate Verilog transformations (in later stages).
Makerchip	Web-based IDE and simulator for writing and testing TL-Verilog designs.

8. Future Work

My primary focus of this GSoC project is to enhance and apply prompt-based flow for converting Verilog to TL-Verilog. I also intend to contribute meaningfully beyond the program period. Below are the future directions I plan to pursue beyond GSOC:

1. Convert Large and Hierarchical IPs
2. Create & Maintain a Public TL-Verilog Library

9.Resume & Portfolio

-  **Personal Website:**
yeshwanthreddykatta.com
-  **GitHub Profile:**
github.com/yeshwanthkattta
-  **LinkedIn Profile:**
linkedin.com/in/katta-yeshwanth-reddy